

Team Project

```
$ echo "Data Sciences Institute"
```

Today

1. Team Project Overview
2. Developing a Project Plan
3. Project Goals and Requirements
4. Working as a Team

Team Project Overview

Goal: To improve your portfolio by showcasing your ability to deliver real-world value to an employer.

Why This Module Matters

- This module is designed to help you create a high-impact project that showcases your skills in data science/machine learning and your ability to work in a team setting.
- Your project will:
 - Demonstrate your technical abilities and problem-solving skills.
 - Serve as a portfolio piece for recruiters, hiring managers, and data science professionals.
 - Showcase what you've learned throughout the certificate.
- Once completed and refined based on feedback, you can add it to your resume, GitHub, and LinkedIn to increase your professional visibility.

Zooming Out - What is a Good Portfolio?

A good portfolio showcases your best work and highlights skills that are highly valued by employers in data science and machine learning. It demonstrates your ability to handle real-world tasks, making you an attractive candidate for employment.

Personalize Your Portfolio:

- **Keep it streamlined:** As you work on your project, ensure that any unused files or folders are deleted to keep your repository clean and professional.
- **Highlight your unique skills:** This personalization shows employers that you're not just completing assignments but are engaged and innovating on your projects.

Selecting Projects That Showcase Your Skills

Choosing the right projects for your portfolio is crucial. Each project should:

- **Solve Real Problems with Real Data:** Use actual datasets to address genuine issues.
- **Demonstrate Industry Relevance:** Select projects that are pertinent to your specific area of interest, such as marketing analytics for marketers or predictive maintenance for engineers.
- **Provide Actionable Insights:** Focus on projects that deliver clear, practical outcomes that demonstrate your ability to impact real-world scenarios.

These criteria ensure that your projects not only highlight your technical skills but also your understanding of and adaptability to industry-specific challenges, making you a valuable candidate to potential employers.

Demonstrating Technical and Business Skills

Success in an industry setting requires expertise in both technical and "soft" skills such as communication and presenting your work.

- **Demonstrate Coding Proficiency:** All projects should showcase your ability to write clean, efficient, and well-documented code.
- **Showcase Communication Skills:** All projects should effectively communicate complex data-driven insights in a non-technical way.

Your ability to document code clearly and communicate effectively can drastically reduce onboarding times for new team members and help non-technical stakeholders make informed decisions, thereby enhancing overall project success and team efficiency.

Types of Projects to Include in Your Portfolio

Including a diverse range of projects in your portfolio can significantly enhance your appeal to potential employers. Consider including a variety of project types to demonstrate the breadth and depth of your data science skills:

1. **Data Cleaning Project:** Show your ability to prepare data for analysis.
2. **Data Storytelling and Visualization Project:** Highlight your skills in interpreting and presenting data in compelling ways.
3. **ML Modeling:** Demonstrate your proficiency in building and tuning models.
4. **Group Project:** Showcase your teamwork and collaboration skills.

For more, read "[How to Create a Project Portfolio for Data Science Job Applications](#)"

Showcase Projects That Use Diverse Data Types

Enhance your projects by effectively using a mix of structured, unstructured, and time series data:

- **Structured Data:** Employ databases for clear, quantifiable insights.
- **Unstructured Data:** Add depth with text, images, or videos.
- **Time Series Data:** Utilize data in sequential order for trend analysis and forecasting.

Select data types strategically to align with your project's objectives. Ensure each type contributes to a clear and coherent narrative, avoiding unnecessary complexity to maintain focus and utility.

Developing a Project Plan

Developing a Project Plan

1. Understand the business context
2. Identify an opportunity
3. Scope your analysis
4. Develop your solution
5. Present clear results and recommendations

Developing a Project Plan

1. **Understand the business context:** "We want to increase customer engagement."
2. **Identify an opportunity:** "Customer segmentation will enable targeted marketing campaigns, which should increase customer engagement."
3. **Scope your analysis:** "We will use feature engineering to create segments based on purchase history and demographic data."
4. **Develop your solution:** "We will use k-means clustering with Python, and determine the optimal number of clusters using the elbow method."
5. **Present clear results and recommendations:** "We predict that applying targeted marketing based on 5 different consumer profiles will increase customer engagement by 10%."

Developing a Project Plan

1. Understand the business context.
2. Identify an opportunity.
3. Scope your analysis.
4. Develop your solution.
5. Present clear results and recommendations.

To keep in mind:

- Are your insights *actionable*?
- How robust is your analysis? *Quantify!*
- What are the *risks* and *unknowns*?

Project Goals

Project Goals

1. **Business Relevance** → Your project should address a real problem or opportunity in your chosen industry, and your results should provide concrete recommendations for your stakeholders.
2. **Good Code & Structure** → Code should be well-commented, clean, and follow a logical structure. It should be easy to read and maintain.
3. **Strong Documentation & Presentation** → The README should be clear, well-written, and explain the dataset, findings, and methodology. The project should be easy to understand for both technical and non-technical reviewers.

Project Goals

- 4. **Application of Module Teachings** → Projects should showcase key technical skills, such as regression modeling, deep learning models, data visualizations, or strong analysis of sampling techniques.
- 5. **Effective Team Collaboration** → Teams should follow best practices for Git (small commits, branches, pull requests) and actively participate in stand-ups and progress updates.

Project Requirements

Final Project Submission

Your final project will be evaluated on the following criteria:

1. Each team member must have created a pull request, and reviewed and merged a different pull request.
2. (a) For **Data Science** teams, your project must include visualizations that present new insights into the chosen dataset.

(b) For **Machine Learning** teams, your project must include at least one machine learning model that you have developed and implemented to obtain new insights.

Final Project Submission

3. Each project's README should describe the project objective, the final outcome of the project, the key business takeaways, and describe your team's approach to working collaboratively. It should also demonstrate thoughtful consideration of the guiding questions listed in this repository's README.

Final Project Submission

4. Your project should be ***completely reproducible***. This means that anyone (e.g. the instructional team for marking, or potential employers for evaluating your work) should be able to completely recreate your analysis and results based on the instructions in your repository. This includes any software and data dependencies. You should test this by trying to regenerate your results from scratch.

Final Project Submission

5. Each team member must record a 3-5 minute video reflecting on your experience. You may each choose where to host your own video, however *it should be public and a link to each team member's video should be included in your project README*. This video is meant to be an asset to your portfolio, and should be available for prospective employers. Your videos should answer the following questions:

- What did you learn?
- What challenges did you face?
- How did you overcome those challenges?
- If you had more time, what would you add?
- What strengths do you bring to a team environment?

Expectations for Week 1

After Week 1, you will only be evaluated on your project proposal, which must be in your project's README file. It should include:

1. The business motivation for your project.
2. Which dataset you have chosen to use.
3. Risks or unknowns that you have identified.
4. How you will approach the analysis.
5. Breakdown of roles/tasks assigned to each team member.

Project Examples

- [Customer Purchasing Behaviours](#) (Data Science)
- [Car Sales Analysis](#) (Data Science)
- [Obesity Estimation](#) (Machine Learning)
- [Insights from Bank Marketing](#) (Machine Learning)
- [TTC Streetcar Delay Analysis](#) (Data Science)
- [Tomato Disease Predictor](#) (Machine Learning - Independent Project)

Working as a Team

Effective GitHub Repository Organization

- Ensure your GitHub repository is neatly organized; avoid unused or empty folders and ensure each folder has a clear purpose.
- Use READMEs in key folders (like `src` , `iac` , `backend` , `frontend` , etc.) to detail their contents and purpose, aiding clarity for complex sections.
- Avoid READMEs in simple folders (like `images` , `docs` , etc.), unless there's specific information that needs to be explained.

Managing your Repository as a Team

Initialization

- **One** team member should create your repository.
- Every other member should **clone** that repository into their local development environment (e.g. laptop).
- ***Do not create different forks of your project.*** This will make collaboration more difficult. Instead, once the project is completed, each team member may create a fork if they wish.

Managing your Repository as a Team

Workflow for feature development

- Do not commit directly to the `main` branch!
- Each feature should be developed in its own branch, then merged into `main` when it has been completed and tested.

Managing your Repository as a Team

Workflow for feature development

1. Pull the latest code from the remote `main` branch to your local `main` branch.
2. Create a new branch from `main`, named according to the feature you are working on.
3. As you develop your feature, make sure you are regularly committing your changes and pushing them to the remote feature branch.
4. Once you have finished developing and testing, create a Pull Request to merge the feature branch into the `main` branch.
5. Make sure another team member is assigned to review and merge the PR.

Merge Conflicts

It is likely that you will encounter merge conflicts when combining your work as a team. Make sure you've reviewed the Git material regarding how to resolve merge conflicts properly!

Working as a Team

Documentation

Crafting a Comprehensive Main README File

- **Purpose & Overview:** Introduce the project with essential details, concise description and a project objective.
- **Goals & Objectives:** Articulate what the project aims to achieve.
- **Techniques & Technologies:** Highlight the tools and methods used.
- **Key Findings & Instructions:** Summarize outcomes and provide setup instructions.
- **Visuals & Credits:** Enhance with visuals; acknowledge contributors.



Writing Clear Documentation (README & Comments)

- **While you can have multiple README files in your repository, include only one README file per folder to keep things clean and simple.**
 - This structure showcases your ability to manage and present complex information effectively, increasing your appeal to potential employers.
 - Creating READMEs in subfolders lets you keep your main README focused. Remember, your main README should be a summary that anyone can understand and should not take longer than ~5 minutes to read.
 - Link to your subfolder READMEs from the main README so readers know where to go for more information.
- **Effective commenting enhances code readability and maintainability, crucial for collaborative environments.**

Writing Clear Documentation (README & Comments)

If someone with no technical background reads your code, they should get the essence of what you are doing and how the code flows. This practice not only aids in understanding but also facilitates smoother transitions and updates within team projects.

 [Best practices for writing code comments](#)

Example



```
function calculateTotal(price, quantity) {  
    return price * quantity;  
}  
  
let totalPrice = calculateTotal(25, 5);  
console.log(totalPrice); // Output: 125
```



```
// Calculates the total cost by multiplying the price per item with the quantity  
function calculateTotal(price, quantity) {  
    return price * quantity;  
}  
  
// Example usage: Calculates the total price for 5 items at $25 each by multiplying the price per item ($25)  
// with the quantity (5), and stores the result in the totalPrice variable.  
let totalPrice = calculateTotal(25, 5);  
console.log(totalPrice); // Output: 125
```

Project Structure

- **One** person from each team must host the primary Git repository. Other team members should clone that repository to work on it.
- There is no hard requirement for the folder structure of your project, but best practices should be followed (see [Team Project Repository](#) for examples).
- Take advantage of multiple README files to keep your main README concise!

Working as a Team

The following two weeks will fly by quickly! How will you ensure that you can accomplish everything you need to as a team in this timeframe?

Working as a Team

1. **Clear objectives and milestones.** Milestones should be set working backward from the final project goal and deadline.
2. **Accountability.** Every action item should have someone assigned (the assigned person can change if workloads become unbalanced).
3. **Communication of roadblocks.** It is just as important to communicate roadblocks and failures as successes. The faster problems are identified, the faster they can be addressed.

Working as a Team

4. **Tracking.** Tracking your tasks is necessary to objectively assess your progress as a team (using software such as [GitHub Projects](#) is helpful). The data-driven mindset applies here too! If you fall behind, update your plan accordingly.
5. **Documentation.** Team members should be able to understand and continue each others' work where necessary.

Module Schedule

Day 1 (Tues): Content delivery, co-work.

Day 2 (Wed): Case study, content delivery, co-work.

Day 3 (Thurs): Review, co-work + standups.

Day 4 (Fri): Co-work.

Day 5 (Sat): Co-work.

Module Schedule

Day 6 (Tues): Content delivery, co-work + standups.

Day 7 (Wed): Co-work + standups.

Day 8 (Thurs): Co-work + standups.

Day 9 (Fri): Co-work.

Day 10 (Sat): Project showcase.

Goals for Today

- Pick an industry.
- Start developing a business case.
- Choose your dataset.

Questions?